ADR-0003: Modular monolith on Java 25 + PostgreSQL/pgvector

Status: Accepted **Date:** 2026-06-01 **Deciders:** Eng lead

Context

Small team, early stage, bounded scope (compare 1–4 developers, tiny data volume). We need clear internal boundaries — so a fragile data-collection module can't sink the app — without distributed-systems overhead. The team is a Java/Spring shop and has chosen **Java 25 (current LTS)**. We need durable storage, time-aware data (append-only observations), and vector search for RAG.

Decision

Build a **modular monolith** with **Spring Boot 3.x + Spring Modulith** on **Java 25 LTS**, persisted in a single **PostgreSQL** instance with **pgvector** for embeddings. Modules: `shared`, `catalog`, `collection`, `userinput`, `reconciliation`, `analytics`, `intelligence`, `api`, with dependency rules enforced by a Spring Modulith verification test. Use only **GA** Java 25 features (Virtual Threads, Scoped Values, Stream Gatherers, pattern matching); avoid preview APIs (e.g. Structured Concurrency) in production. Cross-module side effects use Spring application events. H2 is for tests only.

Options Considered

Option A: Microservices

Dimension	Assessment
Complexity	High (network, deploy, data consistency)
Cost	High
Scalability	High (unneeded at this volume)
Team familiarity	Medium

Pros: Independent scaling/deploy. **Cons:** Massive overhead for a tiny-volume, small-team product; premature.

Option B: Modular monolith (Spring Modulith) — chosen

Dimension	Assessment
Complexity	Medium
Cost	Low
Scalability	Sufficient; a module can later be extracted to a service
Team familiarity	High

Pros: Enforced boundaries now, single deploy, simple ops; split later only if load demands. **Cons:** All modules share a process/datastore; discipline relies on the verification test.

Storage: PostgreSQL + pgvector vs H2 + in-memory vector (v1 plan)

PostgreSQL gives durable, concurrent, time-series-capable storage; pgvector keeps RAG in the **same** datastore (persistent, auditable). H2/in-memory lose data on restart and can't be audited.

Trade-off Analysis

We accept shared-process coupling (mitigated by Modulith boundary tests) to gain simplicity, low cost, and speed. One datastore (Postgres+pgvector) avoids operational sprawl. Java 25 LTS gives a long support window and ideal GA concurrency for targeted parallel collection.

Consequences

- **Easier:** local dev, deployment, refactoring across boundaries, testing with Testcontainers.
- **Harder:** must respect module rules (test-enforced); a future service split needs deliberate work.
- **Revisit:** extract `collection` to its own service if source volume grows; add a broker (Kafka/Rabbit) only when scheduled/streaming volume justifies it; Redis only when a real cache need appears.

Action Items

1. ☐ Scaffold the 8 modules with `@ApplicationModule(allowedDependencies = …)` package-info files.
2. ☐ Add `ModularityTests` running `ApplicationModules.verify()`.
3. ☐ Flyway-managed Postgres schema; H2 only in tests; Testcontainers for DB integration tests.
4. ☐ Use Virtual Threads for collection fan-out; keep Structured Concurrency out until GA.